
SOFTWARE REQUIREMENTS SPECIFICATION

for

ML Inference for ARM64 Devices

Version 3.0

Prepared by:

Piyush Singh Bhati (B24CS1101)
Baviskar Darpan Chandrashekhar (B24CS1020)
Harshit (B24CS1031)
Aashma (B24CS1002)

Alpha Managers:

Piyush Singh Bhati (Technical Lead)
Baviskar Darpan Chandrashekhar (Integration Lead)

Submitted to: Dr. Romi Banerjee

March 18, 2026

Contents

1	Introduction	4
1.1	Purpose	4
1.2	Intended Audience and Reading Suggestions	4
1.3	Project Scope	4
1.3.1	Included in Scope	4
1.3.2	Excluded from Scope	4
1.4	Document Structure	5
2	Overall Description	6
2.1	Product Perspective	6
2.2	Architectural Decisions	6
2.2.1	ARM64 Selection	6
2.2.2	Unikernel Architecture	6
2.2.3	Cooperative Execution Model	6
2.3	Hardware-Induced Timing Variations	7
2.4	User Classes and Characteristics	7
2.5	Product Functions	8
2.5.1	Hardware Initialization and Management	8
2.5.2	Graph Processing Pipeline	8
2.5.3	Execution Engine	8
2.5.4	Memory Management	8
2.5.5	Communication Interface	8
2.5.6	Performance Monitoring	8
2.6	Operating Environment	9
2.7	Design Principles	10
2.8	Assumptions and Dependencies	10
3	System Features	11
3.1	Description and Priority	11
3.1.1	Critical Priority Features	11
3.1.2	High Priority Features	11
3.1.3	Medium Priority Features	11
3.1.4	Low Priority Features	12
3.2	Functional Requirements	12
3.2.1	Hardware Management Requirements	12
3.2.2	Graph Processing Requirements	12
3.2.3	Execution Requirements	12
3.2.4	Memory Requirements	13
3.2.5	Communication Requirements	14
3.2.6	Performance Monitoring Requirements	14
4	Other Nonfunctional Requirements	16
4.1	Performance Requirements	16
4.2	Predictability Requirements	16

4.3	Security Requirements	16
4.4	Reliability Requirements	17
4.5	Power Requirements	17
4.6	Software Quality Attributes	17
4.7	Business Rules	17
5	Development Methodology	19
5.1	Team Structure	19
5.2	Sprint Plan and Timeline (15 January – 10 April)	19
5.2.1	Sprint Logic and Dependencies	20
5.3	Current Sprint Backlog and Alpha Progression	20
5.3.1	Risk Management	21
5.4	Project Management Tools	22
5.5	Design and Coding Standards	22
5.5.1	Function Naming Pattern	22
5.5.2	File Organization	22
5.5.3	Comment Standards	22
5.5.4	Error Handling Pattern	23
6	Other Requirements	24
6.1	Documentation Requirements	24
6.2	Development Constraints	24
6.3	Compliance Requirements	24
6.4	Verification and Validation Strategy	24
7	Traceability Matrix	26
7.1	SRS Progression	26
7.2	UML Diagram Progression	27
7.3	Implementation Status	27
7.4	Alpha Card Progression	28
7.5	Test Case Status	29
7.6	Sprint Traceability	29
8	Test Case Documentation	31
8.1	Overview	31
8.2	Test Environment	31
8.3	Team Assignments	31
8.4	Coverage Summary	32
8.5	Requirement Traceability	32
8.6	Appendices	33
8.6.1	Appendix A: Glossary	33
8.6.2	Appendix B: Acronyms	33
8.6.3	Appendix C: Requirement Templates	33
8.6.4	Appendix D: Performance Analysis Summary	34
8.6.5	Appendix E: Research Basis for Architectural Decisions	34
8.6.6	Appendix F: UART Communication Protocol	34
8.6.7	Appendix G: ONNX Operator Support Matrix	35
8.6.8	Appendix H: Hardware Platform Specifications	35

1 Introduction

Purpose

This document outlines the requirements for a specialized unikernel operating system designed specifically for machine learning inference on ARM64-based embedded platforms. The system represents a departure from general-purpose operating systems by focusing exclusively on ML workload execution, aiming to provide minimal overhead and predictable performance characteristics suitable for edge computing applications [1]. This SRS serves as a blueprint for development and a basis for verification and validation, incorporating evidence from performance analysis of unikernels versus traditional systems.

Intended Audience and Reading Suggestions

This document is intended for several groups:

- **Developers:** For implementation guidance and architectural understanding
- **System Architects:** To understand design decisions and trade-offs
- **Academic Reviewers:** For technical evaluation and feedback
- **Quality Assurance Teams:** To develop comprehensive test plans
- **Project Stakeholders:** For progress tracking and milestone verification

Project Scope

This project focuses on developing a minimal unikernel operating system optimized for ML inference execution on ARM64 processors, with well-defined boundaries to maintain focus and feasibility.

Included in Scope

- ARM64 platform initialization and hardware management
- ONNX computation graph ingestion and validation (static shapes only)
- Graph optimization passes including static scheduling and memory planning
- Pre-execution memory allocation for inference workloads
- Single-threaded execution engine with cooperative task execution
- Basic I/O interfaces for model loading and result retrieval
- ARM64-specific performance optimizations (NEON SIMD, cache awareness)
- Unikernel architecture with single address space design [2]
- Performance monitoring and telemetry collection

Excluded from Scope

- POSIX compatibility or general system call interfaces
- Multi-user or multi-process execution environments
- Dynamic program loading and execution
- General-purpose filesystems and storage management

- Interactive user interfaces or shells
- ML training capabilities
- Dynamic model adaptation or runtime graph modification
- Dynamic shape support in ONNX graphs
- Comprehensive networking stacks beyond basic communication

The system is designed as inference firmware rather than a general-purpose OS, targeting specialized edge AI applications where predictable performance is beneficial.

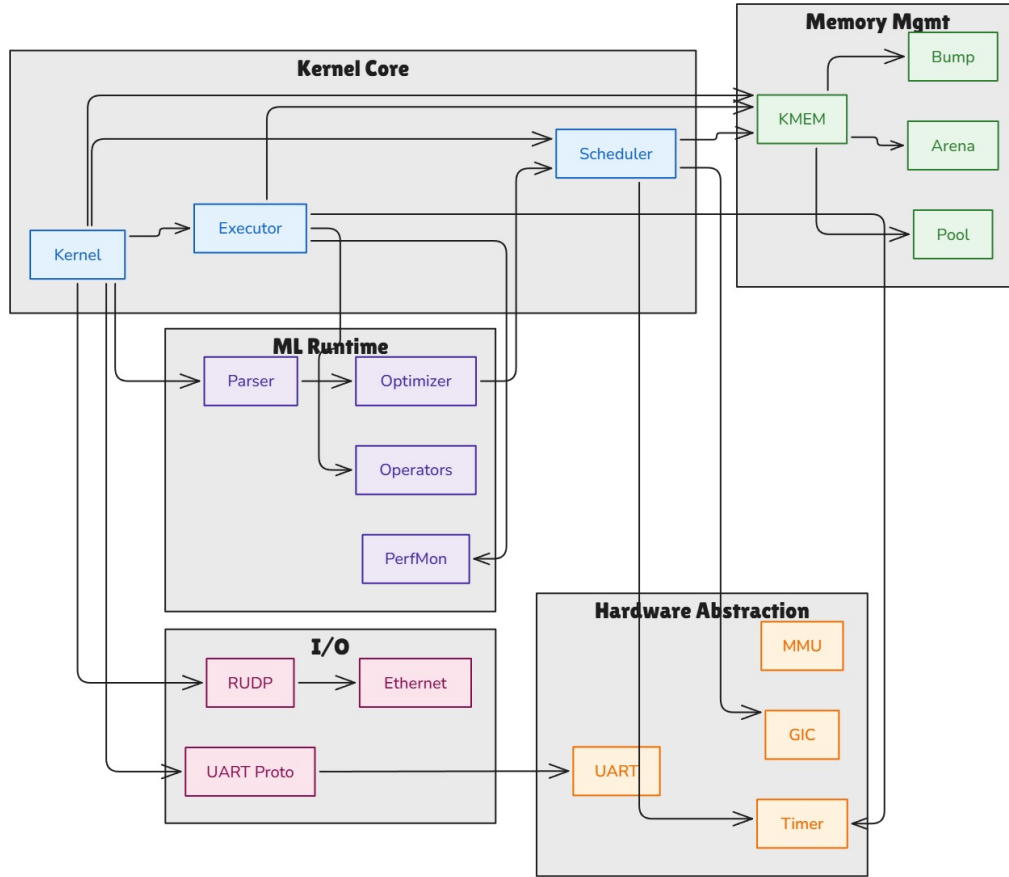


Figure 1.1: Component Diagram: System architecture showing core components and their interactions. The MiniOS unikernel consists of HAL (UART, MMU, GIC, Timer), Kernel Core (memory manager, cooperative scheduler), ONNX Runtime (graph processing, execution engine), and Communication Interfaces (UART protocol, RUDP).

Document Structure

This SRS follows IEEE 830-1998 guidelines for software requirements specifications, organized into seven chapters covering introduction, overall description, system features, nonfunctional requirements, development methodology, other requirements, and appendices with supporting information.

2 Overall Description

Product Perspective

Traditional operating systems like Linux introduce abstraction layers that, while useful for general computing, can introduce overhead for ML inference workloads. Research demonstrates that unikernels can achieve comparable throughput to Linux VMs while offering faster boot times, smaller memory footprints, and reduced attack surfaces [1, 3]. This project creates a specialized unikernel that optimizes the software stack for ML inference on ARM64, treating computation graphs as primary execution units.

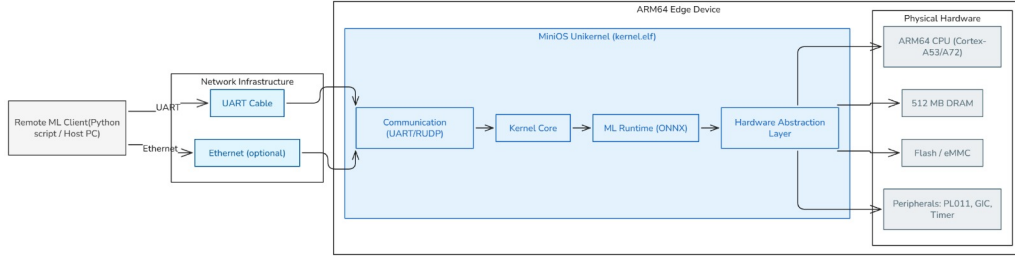


Figure 2.1: Deployment Diagram: System deployment context showing the unikernel running on ARM64 edge hardware (QEMU virt or physical Cortex-A53/A72), communicating with remote ML clients through UART or Ethernet using the RUDP protocol.

Architectural Decisions

ARM64 Selection

- **Decision:** ARM64 (AArch64) over ARM32 (AArch32)
- **Rationale:** Larger register file (31 general-purpose registers), better NEON/SIMD utilization, higher throughput for tensor operations, and support for larger memory addressing [4]
- **Impact:** Enables performance-critical inference and future scalability
- **Limitation:** Requires 64-bit hardware; excludes older ARMv7 devices

Unikernel Architecture

- **Decision:** Unikernel approach over general-purpose OS
- **Rationale:** Single application+kernel image eliminates user/kernel boundary, reduces context switching overhead, and minimizes attack surface
- **Performance Evidence:** Unikernels demonstrate reduced latency variance and higher deployment density in cloud environments [10, 11]
- **Limitation:** Single application focus restricts general-purpose use

Cooperative Execution Model

- **Definition:** Static execution schedule determined at graph compilation time with cooperative task execution. The system uses dependency-based execution where operators run to completion based on a statically analyzed dependency graph, with interrupts handled only

at predefined safe points.

- **Rationale:** Reduces timer interrupt overhead and context switching, providing more predictable timing characteristics for inference applications [6]
- **Implementation:** Graph operators are analyzed for dependencies; a linear execution order is generated; this order is followed with minimal interruption during inference.
- **Note:** This is not true real-time deterministic execution but provides improved predictability over general-purpose scheduling.

Hardware-Induced Timing Variations

The system acknowledges that complete hardware-level determinism is unattainable on standard ARM64 processors due to:

- Cache behavior variations (associativity conflicts, prefetching)
- Dynamic voltage and frequency scaling (DVFS)
- Memory controller arbitration
- Out-of-order execution in Cortex-A7x/A8x processors

The system mitigates these through:

- Static memory allocation to minimize cache conflicts
- Execution timing with tolerance margins
- Performance monitoring to detect significant variations
- Optional frequency locking on supported hardware

User Classes and Characteristics

The system will serve several distinct user groups:

- **System Integrators:** Professionals integrating the OS into embedded devices for AI capabilities
- **ML Engineers:** Developers preparing and optimizing models for edge deployment
- **Deployment Specialists:** Personnel responsible for deploying and maintaining inference systems
- **Hardware Manufacturers:** Companies designing custom AI accelerators or embedded platforms

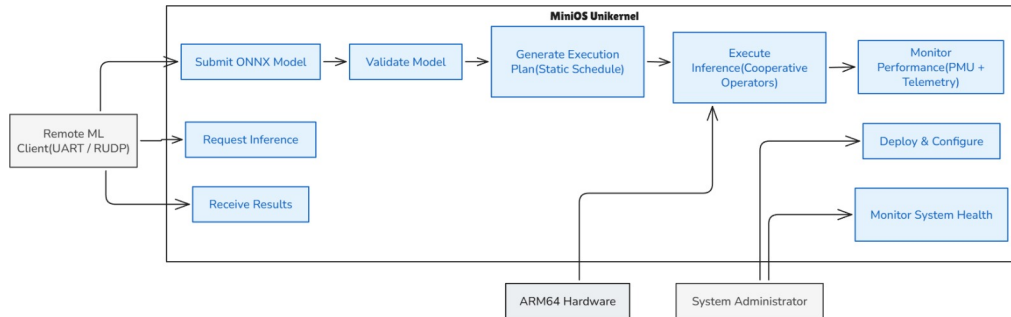


Figure 2.2: Use Case Diagram: System interactions showing how different actors (Remote ML Client, System Administrator) interact with the unikernel through use cases like submitting models (UART/RUDP), requesting inference, retrieving results, and monitoring system health.

Product Functions

The system provides several core functional areas:

Hardware Initialization and Management

- ARM64 processor initialization and boot sequence management
- Memory Management Unit configuration and page table setup
- Cache management and optimization strategies
- Interrupt controller and timer initialization
- Hardware abstraction for ARM64 architecture variants

Graph Processing Pipeline

- ONNX computation graph ingestion through standardized interfaces
- Graph validation for safety and compatibility (static shapes only)
- Optimization passes including operator fusion and static scheduling
- Operator dependency analysis and resolution
- Static execution plan generation

Execution Engine

- Cooperative execution based on static schedules
- Memory access pattern optimization
- ARM64 NEON SIMD instruction utilization [5]
- Execution monitoring and timing measurement

Memory Management

- Whole-graph memory planning and pre-execution allocation
- Tensor lifetime analysis for memory reuse
- Cache-aware memory layout optimization
- Memory protection for critical system regions

Communication Interface

- Minimal graph input interface via UART and RUDP
- Inference result output mechanisms
- System status and health monitoring
- Configuration parameter management

Performance Monitoring

- Execution timing measurement and telemetry
- Memory usage statistics and cache performance
- Energy consumption estimation
- System health and anomaly detection

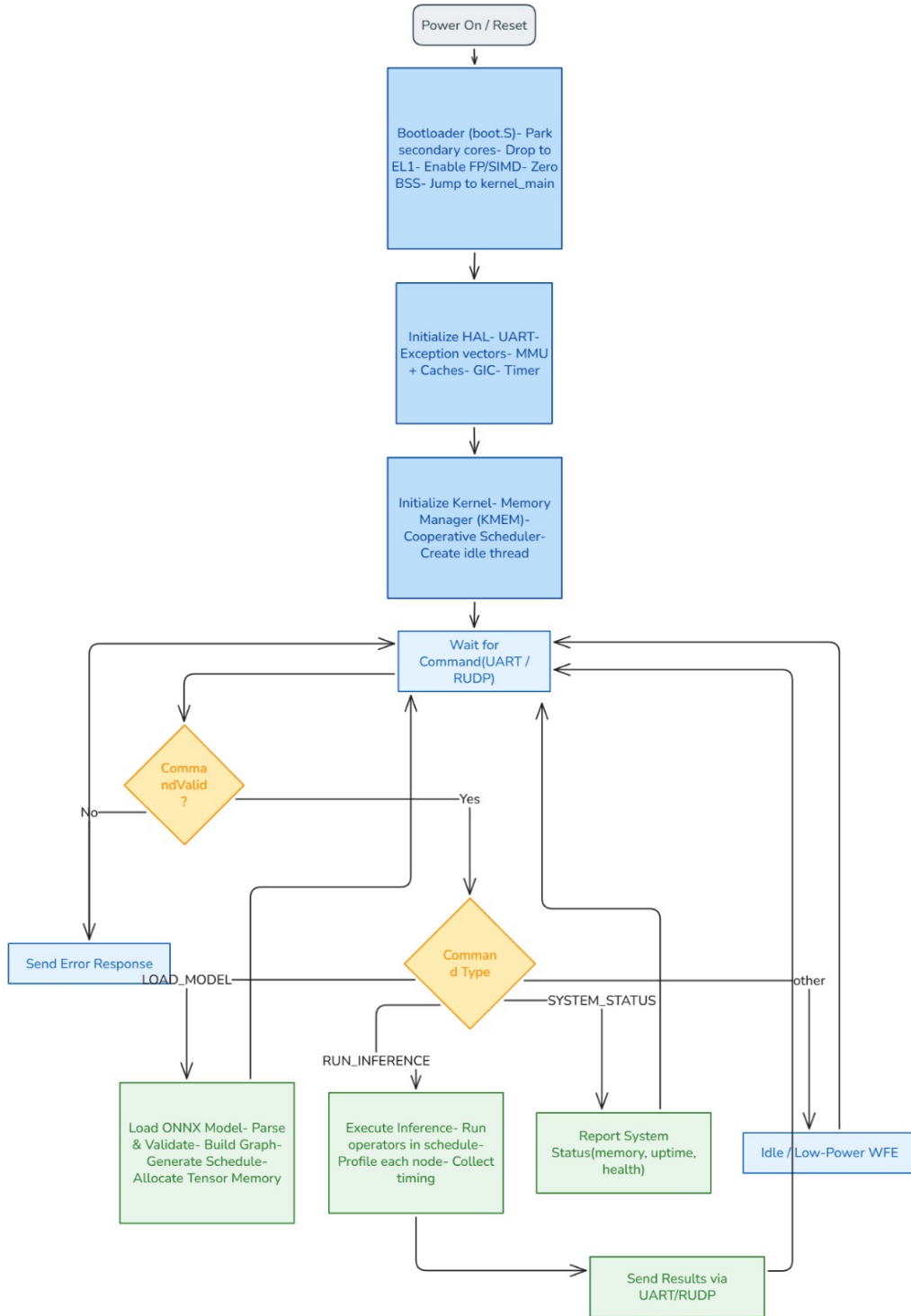


Figure 2.3: Activity Diagram: Operational workflow showing the sequence of activities from boot to inference execution, including model validation, planning, execution, and result delivery. The main loop polls for commands, executes the graph when requested, and yields to background tasks.

Operating Environment

The system is designed for resource-constrained embedded environments:

- **Processor:** ARM64 Cortex-A series processors (Cortex-A53 to Cortex-A78)

- **Memory:** Minimum 64MB RAM (256MB recommended for production use)
- **Storage:** Minimal non-volatile storage for bootloader and configuration (4MB Flash minimum)
- **Communication:** Serial (UART), Ethernet (via RUDP over virtio-net or SMSC LAN9118)
- **Power:** Optimized for battery-operated and power-constrained devices
- **Virtualization:** Compatible with QEMU/KVM for development and testing

Design Principles

The system architecture follows several key design principles:

1. **Graph-Centric Design:** Treating ML computation graphs as primary execution units
2. **Static Optimization:** Leveraging whole-graph knowledge for compile-time optimization
3. **Predictable Execution:** Prioritizing timing predictability over maximum throughput
4. **Minimalism:** Achieving security and reliability through feature elimination
5. **Single Address Space:** Eliminating user/kernel boundary for zero-copy operations
6. **Cooperative Execution:** Reducing preemptive scheduling overhead

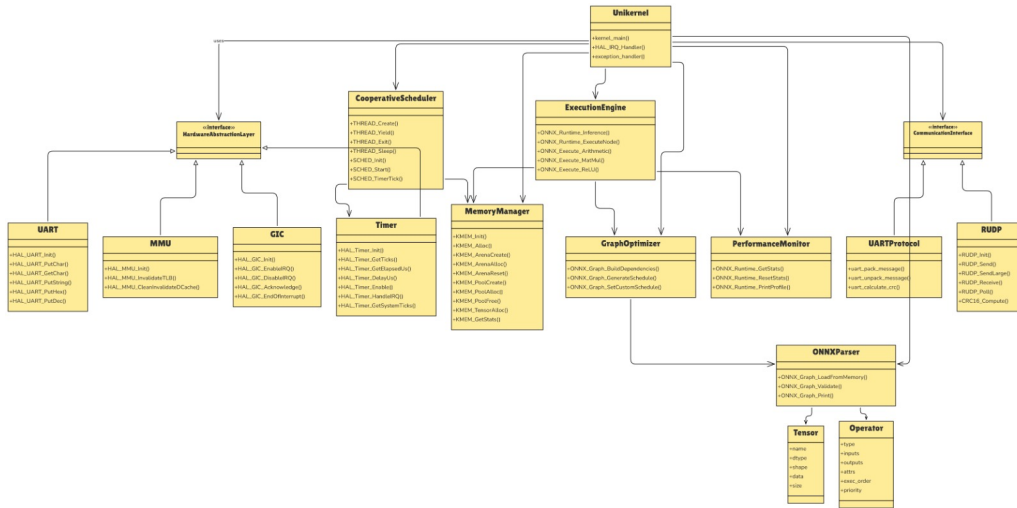


Figure 2.4: Class Diagram: Object relationships showing the main classes in the system: Unikernel, HardwareAbstractionLayer (with subclasses UART, MMU, GIC, Timer), MemoryManager (BumpAllocator, PoolAllocator), ONNXParser, GraphOptimizer, StaticScheduler, ExecutionEngine, PerformanceMonitor, and CommunicationInterface (UARTProtocol, RUDP).

Assumptions and Dependencies

- **A1:** Target hardware provides ARMv8-A compliant processor with NEON extensions
- **A2:** ONNX models use static tensor shapes (no dynamic dimensions)
- **A3:** Inference occurs in controlled environments without adversarial interference
- **A4:** Models fit within available physical memory
- **D1:** Dependence on GCC/Clang ARM64 toolchains for compilation
- **D2:** Dependence on QEMU for development and testing
- **D3:** Dependence on ONNX specification for model compatibility

3 System Features

Description and Priority

Critical Priority Features

1. **ARM64 Hardware Abstraction Layer** (Priority: Critical)
 - Processor initialization and configuration
 - MMU setup for memory protection
 - Cache control and optimization
 - Interrupt controller management
2. **Graph Processing Pipeline** (Priority: Critical)
 - ONNX static graph ingestion and validation
 - Operator dependency analysis using topological sorting
 - Basic optimization passes (operator fusion, constant folding)
 - Static execution plan generation
3. **Memory Management System** (Priority: Critical)
 - Pre-execution memory allocation for entire inference
 - Tensor memory reuse based on lifetime analysis
 - Cache-aware memory layout (64-byte alignment)
 - Memory protection for critical regions

High Priority Features

4. **Execution Engine** (Priority: High)
 - Cooperative execution based on static schedules
 - ARM64 NEON SIMD optimization for common operators
 - Basic error handling without system restart
 - Execution timing measurement
5. **Performance Monitoring System** (Priority: High)
 - Execution timing measurement and telemetry
 - Memory usage statistics and cache performance
 - System health monitoring and anomaly detection
 - Basic energy consumption estimation

Medium Priority Features

6. **Communication Interface** (Priority: Medium)
 - Simple graph input protocol via UART
 - Result output mechanism
 - System status reporting
 - Configuration parameter management

7. **Advanced Optimizations** (Priority: Medium)
 - Extended NEON optimizations for specialized operators
 - Cache prefetching strategies
 - Memory access pattern optimization
 - Operator kernel specialization

Low Priority Features

8. **Multi-Model Support** (Priority: Low)
 - Sequential execution of multiple models
 - Model switching with reinitialization
 - Shared memory optimization across models
9. **Extended Protocol Support** (Priority: Low)
 - Ethernet communication interface
 - Secure boot implementation
 - Remote configuration and monitoring

Functional Requirements

Hardware Management Requirements

1. **FR-001:** The system shall initialize ARM64 processor cores, including system control registers (SCTLR, TCR, MAIR) according to ARMv8-A specification.
2. **FR-002:** The system shall configure the MMU with minimal page tables for required memory regions using 4KB granularity, establishing separate memory regions for code, data, and device memory.
3. **FR-003:** The system shall enable and manage data and instruction caches with write-back policy, providing cache maintenance operations for coherence.
4. **FR-004:** The system shall handle hardware interrupts with configurable priorities, deferring non-critical interrupt processing to safe execution points.
5. **FR-005:** The system shall provide timer services for execution timing with microsecond resolution using ARM Generic Timer.

Graph Processing Requirements

6. **FR-006:** The system shall accept ONNX computation graphs in binary protobuf format (ONNX version 1.8+), supporting a subset of operators defined in Appendix G.
7. **FR-007:** The system shall validate graph integrity, operator compatibility, and tensor dimensions, rejecting graphs with dynamic shapes or unsupported operations.
8. **FR-008:** The system shall perform operator dependency analysis using topological sorting on the computation graph DAG.
9. **FR-009:** The system shall generate static execution schedules based on operator dependencies, with optional operator fusion for performance.
10. **FR-010:** The system shall perform whole-graph memory planning with tensor lifetime analysis, allocating memory before execution begins.

Execution Requirements

11. **FR-011:** The system shall execute operators in dependency order with cooperative execution, running each operator to completion before proceeding to the next.
12. **FR-012:** The system shall measure and record execution timing for each operator, pro-

viding timing statistics for performance analysis.

13. **FR-013:** The system shall optimize for ARM64 NEON SIMD instructions for supported operators (Conv2D, Matrix Multiplication, Element-wise operations).
14. **FR-014:** The system shall provide predictable memory access patterns through static allocation and cache-conscious layout.
15. **FR-015:** The system shall handle errors through predefined error codes and recovery procedures without restarting the inference.

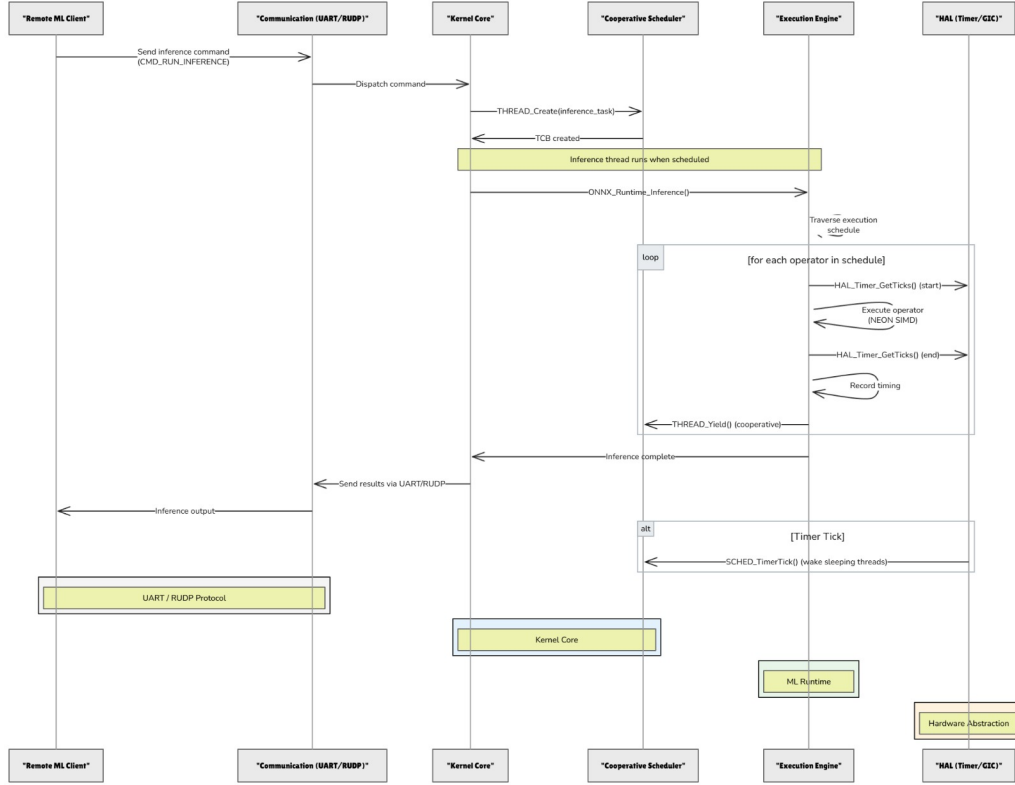


Figure 3.1: Sequence Diagram: Execution flow for an inference request. Client sends command via UART/RUDP, kernel parses, scheduler dispatches graph nodes, operators execute in dependency order, results are returned.

Memory Requirements

16. **FR-016:** The system shall allocate all required memory before execution begins, using a static allocator with fixed-size pools for tensors.
17. **FR-017:** The system shall reuse memory for tensors with non-overlapping lifetimes based on lifetime analysis during graph compilation.
18. **FR-018:** The system shall optimize memory layout for cache performance using 64-byte alignment and cache line awareness.
19. **FR-019:** The system shall protect critical memory regions (code, stack, system data) from corruption using MMU protection attributes.
20. **FR-020:** The system shall provide memory usage statistics including peak usage, fragmentation, and cache performance metrics.

Communication Requirements

21. **FR-021:** The system shall provide a minimal interface for graph input via UART with a simple binary protocol (see Appendix F).
22. **FR-022:** The system shall output inference results via the same UART interface in a defined binary format.
23. **FR-023:** The system shall report system status and health metrics including temperature, voltage, and error counts.
24. **FR-024:** The system shall accept configuration parameters for optimization (cache policy, frequency scaling, etc.).
25. **FR-025:** The system shall validate all inputs for correctness and safety, rejecting malformed or suspicious data.

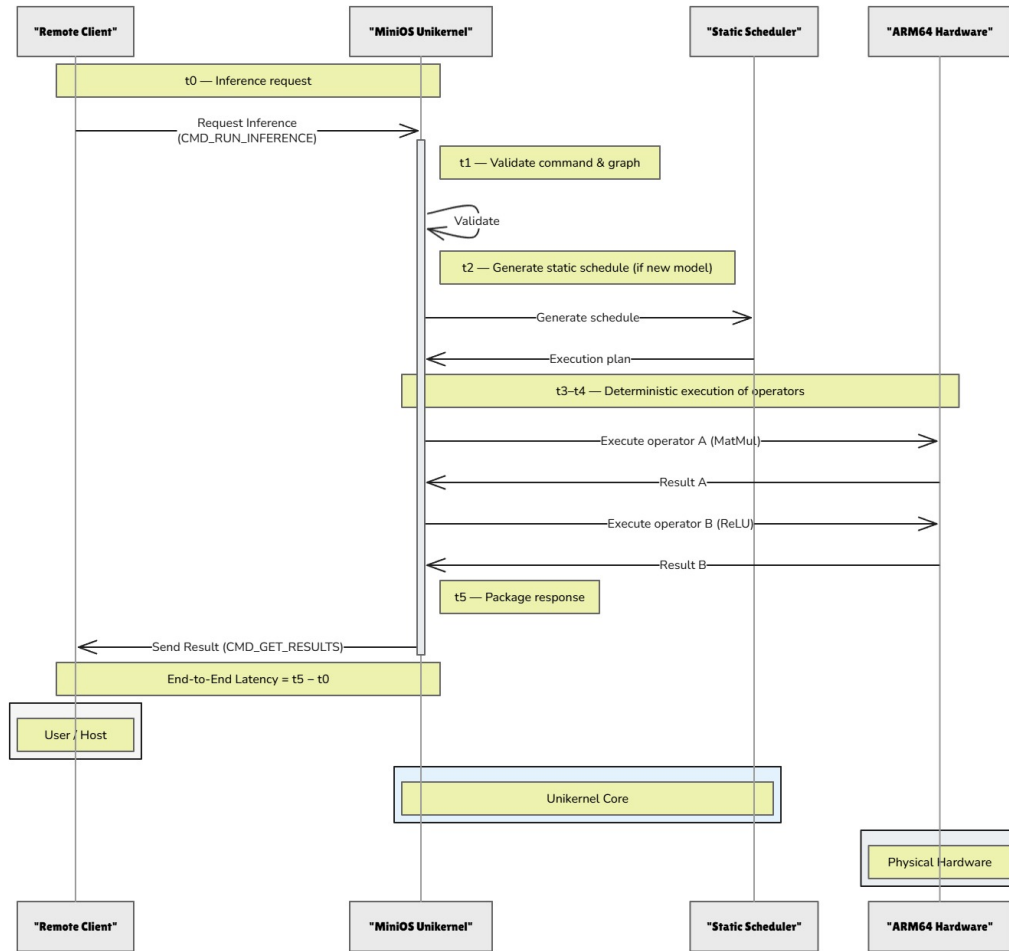


Figure 3.2: Timing Diagram: Temporal relationships during inference execution showing the sequence of events (t0–t5) from request to response with end-to-end latency measurement. Typical times measured in benchmark suite: command reception, graph execution (operators A, B, C), result transmission.

Performance Monitoring Requirements

26. **FR-026:** The system shall measure and record execution timing for each operator using ARM performance counters where available.
27. **FR-027:** The system shall collect memory usage statistics throughout execution including allocation patterns and cache behavior.

28. **FR-028:** The system shall monitor cache hit/miss ratios using ARM performance counters on supported hardware.
29. **FR-029:** The system shall estimate energy consumption based on execution profile and hardware power models.
30. **FR-030:** The system shall detect and report performance anomalies exceeding configurable thresholds.

4 Other Nonfunctional Requirements

Performance Requirements

1. **PR-001:** The system shall achieve inference throughput within 20% of a bare-metal inference implementation on the same hardware for identical models and inputs.
2. **PR-002:** The system overhead (excluding model execution) shall not exceed 10% of total inference time for models with 100+ operators.
3. **PR-003:** Graph optimization passes shall complete within 100ms for graphs up to 1000 operators on a 1GHz Cortex-A53.
4. **PR-004:** Memory allocation shall complete within 10ms for models requiring up to 64MB of tensor memory.
5. **PR-005:** Interrupt latency shall be bounded to less than 50 microseconds for high-priority interrupts.
6. **PR-006:** Boot time shall be less than 200 milliseconds from power-on to inference-ready state on typical hardware.

Predictability Requirements

1. **PDR-001:** Execution time variation shall be less than 15% across 100 identical runs under consistent environmental conditions (temperature, voltage) [7].
2. **PDR-002:** Memory allocation patterns shall be identical for identical graph structures and input sizes.
3. **PDR-003:** Cache behavior shall be managed through static allocation to reduce variance, though complete determinism is not guaranteed due to hardware factors.
4. **PDR-004:** Interrupt handling shall preserve execution predictability through deferred processing at safe points.
5. **PDR-005:** System shall provide empirical execution time bounds for each operator based on measurement-based analysis.

Security Requirements

1. **SR-001:** The system shall validate all input graphs for structural integrity and bounds checking [8].
2. **SR-002:** Memory protection shall prevent unauthorized access to critical regions through MMU configuration.
3. **SR-003:** The system shall not support dynamic code loading or self-modifying code.
4. **SR-004:** Communication interfaces shall include basic integrity checking (CRC) for data validation.
5. **SR-005:** The attack surface shall be minimized by eliminating unused system components and interfaces.

Reliability Requirements

1. **RR-001:** The system shall operate continuously for 72 hours without failure under normal operating conditions.
2. **RR-002:** Memory corruption shall be detected through bounds checking and validation with 95% probability for common error patterns.
3. **RR-003:** The system shall recover from non-fatal errors through predefined recovery procedures without full restart.
4. **RR-004:** All critical operations shall have validation checks with appropriate error reporting.
5. **RR-005:** The system shall provide health status monitoring with configurable reporting intervals.

Power Requirements

1. **PWR-001:** The system shall support sleep modes between inferences to reduce idle power consumption [9].
2. **PWR-002:** Power consumption shall be measurable and reportable through software estimation.
3. **PWR-003:** The system shall optimize for energy efficiency through batch processing and operator fusion.
4. **PWR-004:** Clock gating shall be employed for idle components when supported by hardware.
5. **PWR-005:** Power management operations shall not cause system failure or data corruption.

Software Quality Attributes

- **Maintainability:** Code shall be modular with clear separation of concerns. Documentation shall cover all interfaces and architectural decisions.
- **Testability:** All components shall be unit testable with mock hardware interfaces. System shall support reproducible execution for regression testing.
- **Portability:** Hardware abstraction shall support multiple ARM64 variants with compile-time configuration.
- **Verifiability:** All functional requirements shall be verifiable through testing, inspection, or analysis.
- **Simplicity:** System shall minimize complexity by eliminating unnecessary features and keeping interfaces minimal.

Business Rules

1. **BR-001:** The system shall be licensable under open-source terms for academic and commercial embedded use.
2. **BR-002:** The system shall support ONNX model format standard version 1.8+ with documented limitations.
3. **BR-003:** The system design shall consider safety-critical application requirements for future certification.
4. **BR-004:** The system shall have minimal dependencies on proprietary software or libraries.

5. **BR-005:** The system architecture shall allow extension for custom hardware accelerators through well-defined interfaces.

5 Development Methodology

Team Structure

- **Technical Lead:** Piyush Singh Bhati
 - Responsible for overall architecture and technical decisions
 - Oversees code quality and design patterns
 - Manages integration and testing strategy
 - Primary contact for technical queries
- **Integration Lead:** Baviskar Darpan Chandrashekhar
 - Manages hardware abstraction and platform integration
 - Oversees build system and deployment pipeline
 - Coordinates with other team members on interface definitions
 - Manages toolchain and development environment
- **ML Runtime Developer:** Harshit
 - Implements graph optimization and execution engine
 - Develops tensor operations and NEON optimizations
 - Implements ONNX parser and validator
 - Develops performance monitoring components
- **System Components Developer:** Aashma
 - Implements memory management and communication interfaces
 - Develops hardware abstraction layer components
 - Implements bootloader and initialization code
 - Develops testing infrastructure and documentation

Sprint Plan and Timeline (15 January – 10 April)

The project started on 15 January and is scheduled to complete on 10 April. Work is organised into six two-week sprints. The table below details the objectives and current status as of mid-March.

Table 5.1: Sprint Plan (15 January – 10 April)

Sprint	Dates	Primary Objectives	Status & Key Deliverables
Sprint 1	Jan 15 – Jan 28	Development Environment Setup	Completed – ARM64 toolchain, QEMU, build system, basic UART debug.
Sprint 2	Jan 29 – Feb 11	Bootloader and Basic HAL	Completed – ARM64 boot sequence, exception vectors, MMU init, timer and GIC setup.
Sprint 3	Feb 12 – Feb 25	Memory Management System	Partially complete – Bump allocator and pool allocator designed; tensor lifetime analysis and memory planning in progress.
Sprint 4	Feb 26 – Mar 11	Graph Processing Pipeline	Core complete – ONNX parser, graph validation, dependency analysis, basic operator implementations (Arithmetic, MatMul, ReLU); Conv/Pool stubs remain.
Sprint 5	Mar 12 – Mar 25	Communication & Networking	Completed – UART binary protocol (FR-021/022), RUDP reliable networking (FR-023/025).
Sprint 6	Mar 26 – Apr 10	Integration, Testing & Performance	In progress – Merging branches, end-to-end inference test, full performance benchmarking, documentation finalisation.

Sprint Logic and Dependencies

- Hardware initialization (Sprint 2) is a prerequisite for memory management (Sprint 3).
- Memory management must be ready before graph processing (Sprint 4) because tensor allocation depends on it. However the branch of [feat/onnx](#) contains a separate implementation of ONNX runtime based on a parallel development from an under-developed kernel API. The further developments in the memory management unit will be developed based on the ONNX runtime requirements.
- Communication components (Sprint 5) were developed in parallel and are now being integrated.
- Final integration (Sprint 6) will combine all subsystems and validate against SRS requirements.

Current Sprint Backlog and Alpha Progression

We track project health using the seven Essence alphas. The table below summarises the current state of each alpha and the work in progress as of Sprint 6.

Alpha	Current State	In-Progress / Next Steps
Opportunity	Conceived, Bounded	The need for a lightweight ML inference unikernel is clearly defined and scoped.
Stakeholders	Recognised, Represented	Team, instructor, and potential edge-device users are identified; feedback from reviews incorporated.
Requirements	Conceived, Bounded, Coherent	SRS v3.1 documents all functional and non-functional requirements; traceability matrix maintained.
Software System	Architecture Selected, Demonstrable	Core subsystems (boot, HAL, scheduler, ONNX runtime, RUDP) are implemented and demonstrated individually. Integration into a single bootable image is the current focus.
Work	Initiated, Under Control	Tasks are tracked in Trello; approximately 85% of planned work is complete; remaining items are in Sprint 6.
Way of Working	Principles Established, Tools in Place	Kanban board, GitHub, code reviews, and coding standards are actively used.
Team	Formed, Collaborating	All members are contributing; regular meetings and pair programming sessions occur.

The current sprint (Sprint 6) backlog includes:

- Merging the ‘feat/onnx’, ‘net-protocol’, and ‘Kernel_API’ branches.
- Implementing the static memory allocator (FR-016 through FR-020).
- Completing NEON-optimised Conv2D and Pooling operators.
- Wiring the Kernel API (KAPI) as the unified interface.
- Running full system tests and collecting performance metrics.

Risk Management

- **Technical Risks:**
 - ARM64 hardware complexity may extend HAL development time
 - ONNX parser may require more effort than anticipated
 - NEON optimization may have steep learning curve
- **Mitigation Strategies:**
 - Start with minimal HAL and expand gradually
 - Use existing ONNX libraries as reference
 - Focus on critical operators first (Conv2D, MatMul)
- **Contingency Plan:** Reduce scope to core features if timeline pressure increases

Project Management Tools

- **Primary Tool:** Trello (Kanban Board)
- **Justification:**
 - Visual task management suitable for small team (4 members)
 - Easy tracking of sprint progress and task dependencies
 - Integration with GitHub for code linking
 - Mobile accessibility for on-the-go updates
- **Board Structure:**
 - Backlog: All identified tasks and requirements
 - Sprint Planning: Tasks selected for current sprint
 - In Progress: Actively being worked on
 - Code Review: Completed development, awaiting review
 - Testing: Under integration testing
 - Done: Completed and verified
- **Supporting Tools:**
 - GitHub: Version control and code collaboration
 - Project Link: <https://github.com/Piyush2005-code/MiniOS.git>
 - Google Docs: Shared documentation
 - Project Documentation: <https://docs.google.com/document/d/10xoFz6mBQMf71bza4sphlP8VE7qv/edit>
 - QEMU: ARM64 emulation for testing
 - GCC/Clang: Cross-compilation toolchains

Design and Coding Standards

Function Naming Pattern

```
// ModulePrefix_ActionObject convention
// Examples:
void HAL_UART_Initialize(UART_Config* config);
Status MEM_AllocateTensor(Tensor* tensor, size_t size);
Graph* GRAPH_ParseONNX(const uint8_t* buffer, size_t length);
Status EXEC_RunOperator(Operator* op, Tensor** inputs);
```

File Organization

```
/src
/  hal          # Hardware abstraction layer
/  mem          # Memory management
/  graph        # Graph processing
/  exec         # Execution engine
/  comm         # Communication interfaces
/  perf         # Performance monitoring
/  lib          # Common utilities
/include        # Public headers
/tests         # Unit and integration tests
/build         # Build scripts and configurations
/docs          # Documentation
```

Comment Standards

```
/**
 * @brief Brief description of function purpose
```

```

*
* Detailed description including algorithm overview,
* performance characteristics, and memory requirements.
*
* @param[in] input_param Description of input parameter
* @param[out] output_param Description of output parameter
* @return Status code indicating success or error type
*
* @note Important notes about usage or limitations
* @warning Critical warnings about proper usage
*
* @complexity Time:  $O(n)$ , Space:  $O(1)$ 
*/
Status MODULE_FunctionName(Type input_param, Type* output_param);

```

Error Handling Pattern

```

// All functions return Status enum
typedef enum {
    STATUS_OK = 0,
    STATUS_ERROR_INVALID_ARGUMENT,
    STATUS_ERROR_OUT_OF_MEMORY,
    STATUS_ERROR_EXECUTION_TIMEOUT,
    STATUS_ERROR_HARDWARE_FAULT,
    // ... other error codes
} Status;

// Usage pattern
Status result = FUNCTION_Call();
if (result != STATUS_OK) {
    LOG_Error("Function failed with code: %d", result);
    return result; // Propagate error
}

```

6 Other Requirements

Documentation Requirements

1. **DOC-001:** Complete API documentation for all system interfaces using Doxygen format.
2. **DOC-002:** Hardware integration guide for common ARM64 development boards (Raspberry Pi 3/4, QEMU).
3. **DOC-003:** Performance tuning guide with optimization techniques and configuration options.
4. **DOC-004:** Security guidelines for deployment in sensitive environments.
5. **DOC-005:** Troubleshooting guide for common issues and error recovery procedures.
6. **DOC-006:** Build instructions for cross-compilation and deployment.

Development Constraints

1. **DC-001:** System shall be implemented in C (C11 standard) and ARM64 assembly.
2. **DC-002:** Runtime dynamic memory allocation shall not be used; all allocation occurs before execution.
3. **DC-003:** Total code size shall not exceed 256KB for core system including essential libraries.
4. **DC-004:** Floating-point operations shall use ARM64 hardware FPU where available, with software fallback.
5. **DC-005:** Timing-critical code shall be written for analyzability, with avoidance of complex control flow in hot paths.
6. **DC-006:** Third-party dependencies shall be limited to standard C library subset and compiler runtime.

Compliance Requirements

1. **COMP-001:** Shall comply with ARM Architecture Reference Manual ARMv8-A for AArch64 execution state.
2. **COMP-002:** Shall support ONNX operator standard version 1.8+ with documented subset compliance.
3. **COMP-003:** Shall be compatible with GCC 10+ and Clang 12+ ARM64 toolchains.
4. **COMP-004:** Shall support UART communication following RS-232 standard for basic operation.
5. **COMP-005:** Shall follow MISRA C:2012 guidelines for critical safety-related code sections.

Verification and Validation Strategy

- **Verification Methods:**
 - Inspection: Code reviews, documentation reviews

- Analysis: Static analysis, complexity analysis
- Testing: Unit testing, integration testing, system testing
- Demonstration: End-to-end inference demonstrations
- **Validation Approach:**
 - Functional correctness: Compare inference results with reference implementations
 - Performance validation: Measure against performance requirements
 - Usability validation: Test with sample models and edge cases
- **Acceptance Criteria:**
 - All critical requirements (Priority: Critical) must pass verification
 - 90% of high priority requirements must pass verification
 - System must demonstrate end-to-end inference for at least three sample models

7 Traceability Matrix

The complete implementation traceability matrix, including detailed development tracking, status updates, and team member assignments, is maintained in the following Google Sheets document:

Traceability Matrix URL:

<https://docs.google.com/spreadsheets/d/1DZKQ69UU6-ZY28EB0zRdZdpqSy9iL9XjS6t5NvDDCjw/edit?gid=1492362285#gid=1492362285>

This spreadsheet provides real-time tracking organized into the following columns:

- **Attribute:** Requirement identifier (FR-XXX, PR-XXX, BR-XXX)
- **Requirements:** Functional / Non-functional / Business requirement description
- **Component:** System component where implementation resides
- **Class:** Object-oriented class containing the function
- **Functions:** Specific functions/methods implementing the requirement
- **Start Date of Code:** When implementation began
- **End Date of Code:** When implementation completed
- **Update Date:** Date of any subsequent updates
- **Update Made:** Description of changes made during updates
- **Reason for Update:** Justification for modifications
- **Team Member(s) Designing / Updating Code:** Developer responsible
- **Test / Integration Status Details:** Specific test results and metrics
- **Team Member(s) Testing Code:** QA/Testing personnel

The following sections summarise the progress across different areas of the project, as tracked in the above spreadsheet and associated documents.

SRS Progression

The Software Requirements Specification has evolved through multiple versions, reflecting iterative refinement based on feasibility analysis and stakeholder feedback. Key milestones are summarised in Table 7.1. The full documents are available at:

- **SRS v1.0:** https://github.com/Piyush2005-code/MiniOS/blob/SRS-and-Reports/SRS_Version1.0.pdf
- **SRS v2.0:** https://github.com/Piyush2005-code/MiniOS/blob/SRS-and-Reports/SRS_Version2.0.pdf

Table 7.1: SRS Progression

Date	Updates	Version	Approved by
20 Jan 2026	Initial creation covering full scope: ARM64 HAL, graph processing (dynamic shapes, full ONNX), multi-threaded execution, Ethernet, POSIX compatibility.	v1.0	Piyush Singh Bhati
1–4 Feb 2026	Major revision based on feasibility: removed dynamic shapes, full ONNX, Ethernet, real-time guarantees, POSIX; added static shapes only, operator matrix (Appendix G), UART-only communication, predictability focus, single-threaded cooperative model. Priority classifications introduced.	v2.0	Harshit, Aashma, Piyush, Darpan

UML Diagram Progression

UML diagrams have been updated to reflect architectural decisions. Initial versions (v1.0) and refined versions (v2.0) are available in the repository folders:

- **v1.0:** https://github.com/Piyush2005-code/MiniOS/tree/SRS-and-Reports/images_v1
- **v2.0:** https://github.com/Piyush2005-code/MiniOS/tree/SRS-and-Reports/images_v2

Table 7.2 lists the diagrams and their update history.

Table 7.2: UML Diagram Progression

Date	Diagram Category	Updates
20 Jan 2026	Component, Deployment, Use Case, Activity	Initial creation (v1.0)
21 Jan 2026	Class, Sequence, Timing, Composite	Initial creation (v1.0)
3–4 Feb 2026	All diagrams	Refined component boundaries, simplified actors, added validation steps, deferred interrupt handling, end-to-end latency measurement (v2.0)

Implementation Status

The implementation tracker lists all functional, non-functional, and business requirements along with their completion status. As of 18 March 2026, the following requirements have been fully implemented and tested:

- **Hardware Abstraction:** FR-001 through FR-005, FR-021 (UART, MMU, GIC, Timer)
 - **Complete**

- **Graph Processing:** FR-006 through FR-009, FR-011 through FR-013, FR-015 (ONNX loader, dependency analysis, execution engine, NEON operators) – **Core complete** (Conv/Pool stubs remain)
- **Communication:** FR-021 through FR-025, SR-004, NFR-001–003, DC-002 (UART protocol, RUDP) – **Complete**
- **Memory Management:** FR-016, FR-017, FR-018, FR-020 (bump/arena/pool allocators, tensor alignment) – **Complete** (FR-019 protection pending)
- **Performance Benchmarking:** PR-001, PR-002, PR-006, PDR-001, BR-005 – **Complete**

A detailed breakdown per requirement is maintained in the online spreadsheet. Table 7.3 summarises the counts by priority and subsystem.

Table 7.3: Implementation Summary by Priority

Priority	Total	Completed	Notes
Critical	3 feature groups	3 groups fully implemented	HAL, Graph, Memory (core)
High	2 feature groups	2 groups fully implemented	Execution Engine, Performance Monitoring
Medium	2 feature groups	1 group fully implemented (Communication)	Advanced optimisations pending
Low	2 feature groups	0 groups fully implemented	Multi-model support, extended protocols

Alpha Card Progression

Project health is tracked using the Essence alphas. Table 7.4 shows the current state of each alpha as of 18 March 2026, based on the alpha card progression log.

Table 7.4: Alpha Card Progression

Date	Alpha Concern	Current Card	Status
4 Feb 2026	Customer – Opportunity	4 (Viable)	Completed
18 Feb 2026	Customer – Stakeholders	3 (Involved)	In Progress
4 Feb 2026	Solution – Requirements	3 (Coherent)	Completed
25 Feb 2026	Solution – Software System	2 (Demonstrable)	In Progress
11 Feb 2026	Endeavor – Team	3 (Collaborating)	Completed
25 Feb 2026	Endeavor – Work	3 (Started)	In Progress
18 Feb 2026	Endeavor – Way of Working	3 (In Use)	Completed
2 Mar 2026	Solution – System Integration	2 (Demonstrable)	In Progress
2 Mar 2026	Solution – End-to-End Inference	1 (Concept)	Planned
2 Mar 2026	Solution – Operator Validation	1 (Concept)	Planned
8 Mar 2026	Customer – Demonstrable MVP	2 (Demonstrable)	Planned
12 Mar 2026	Endeavor – Work	3 (Started)	In Progress

The Trello board used for sprint tracking is available at: <https://trello.com/b/5Z1rnMOJ/swegroup12>

Test Case Status

A comprehensive suite of **246 test cases** was implemented and executed across all major subsystems as of 18 March 2026. **All 246 tests passed** on the QEMU virtual platform. Full details are documented in Chapter 8 of this report.

Table 7.5: Test Case Summary (as of 18 March 2026)

Component	Tester	Unit	Comp.	System	Total	
UART Driver	Darpan Baviskar	21	–	–	21	
Timer Driver	Darpan Baviskar	17	–	–	17	
MMU Driver	Darpan Baviskar	17	–	–	17	
Type Definitions	Harshit	17	–	–	17	
Memory Manager	Harshit	45	7	–	52	
Status Codes	Piyush Singh Bhati	6	–	–	6	
Scheduler	Piyush Singh Bhati	54	14	–	68	
Context Switch	Aashma	8	–	–	8	
Exception Handling	Aashma	–	4	–	4	
Kernel API	Aashma	–	4	–	4	
System Boot	Aashma	–	–	6	6	
System Init	Aashma	–	–	5	5	
System API	Aashma	–	–	5	5	
System Benchmark	Aashma	–	–	10	10	
System Memory Stress	Aashma	–	–	3	3	
System Stability	Aashma	–	–	3	3	
Total		185	29	32	246	

Sprint Traceability

The project is organised into six two-week sprints. Table 7.6 summarises the tasks completed in each sprint, with links to the corresponding commit history and Trello cards. Detailed task breakdown is available in the online spreadsheet.

Table 7.6: Sprint Progress Summary

Sprint	Dates	Key Deliverables
Sprint 1	Jan 15–28	Repository setup, SRS v1.0, build system
Sprint 2	Jan 29–Feb 11	Bootloader, HAL (UART, MMU, vectors), basic UART output
Sprint 3	Feb 12–25	Memory manager (bump allocator), UART protocol, benchmarking framework, arch helpers, string library
Sprint 4	Feb 26–Mar 11	GIC, Timer, ONNX loader, graph processing, scheduler algorithms (7 policies), RUDP protocol, Ethernet driver, CRC-16, arena/pool allocators, threading, context switch, full IRQ handler, kapi.h, memory stats
Sprint 5	Mar 12–25	Operator registry, NEON operators (7), integration of all branches, final system testing (ongoing)
Sprint 6	Mar 26–Apr 10	Full system integration, performance optimisation, documentation finalisation (planned)

For the most up-to-date implementation status, please refer to the online spreadsheet which is maintained throughout the development lifecycle. The matrix tracks all functional, non-functional, and business requirements with detailed implementation metrics including through-

put measurements, latency benchmarks, and performance comparisons between Linux and Unikraft implementations.

8 Test Case Documentation

Overview

A comprehensive suite of **246 test cases** was designed, executed, and verified against the functional and non-functional requirements defined in this SRS. Tests cover all 16 system components across three environments: QEMU ARM64 target tests (UART, Timer, MMU, Context Switch, Exceptions, System), and host-side unit tests compiled on x86_64 using the Unity 2.6.2 framework. **All 246 test cases passed** as of 18 March 2026.

The full test case documentation — including individual test procedures, prerequisites, expected results, and execution evidence — is maintained in the project traceability spreadsheet and the companion Test Case Documentation report (MiniOS-TST-001 v1.0).

Test Environment

Table 8.1: Test Execution Environment

Parameter	Details
Emulator	QEMU 5.0+, machine virt, cpu cortex-a53 (Appendix H)
Architecture	AArch64 (ARMv8-A), bare metal, EL1
Toolchain	GCC 10+ or Clang 12+, aarch64-linux-gnu, -nostdlib -ffreestanding (DC-001, COMP-003)
Language	C11 standard + ARM64 assembly; core ≤ 256 KB (DC-001, DC-003)
Host Framework	Unity 2.6.2, gcc -std=c11, x86_64 Linux (Ubuntu 24.04)
UART	PL011 at 0x09000000; 115200 bps, 8N1 (Appendix F)
Timer	ARM Generic Timer (CNTP), ≈ 62.5 MHz (FR-005)
GIC	GICv2, IRQ_TIMER_PHYS = 30 (Appendix H)
Target HW	Raspberry Pi 3/4 (Cortex-A53/A72), NVIDIA Jetson Nano (Cortex-A57)

Team Assignments

Table 8.2: Test Responsibility Matrix

ID	Member	Role	Domain	Cases	Period
B24CS1020	Darpan Baviskar	Integration Lead	HAL: UART, Timer, MMU	55	Mar 10–12
B24CS1031	Harshit	ML Runtime Developer	Type Definitions, Memory Manager	69	Mar 10–14
B24CS1101	Piyush Singh Bhati	Technical Lead	Status Codes, Scheduler	74	Mar 12–18
B24CS1002	Aashma	System Components Developer	Context, Exceptions, KAPI, System	48	Mar 13–18
Total				246	Mar 10–18

Coverage Summary

Table 8.3: Test Coverage by Component (All Tests: PASS)

Component	Tester	Unit	Comp.	System	Total
UART Driver	Darpan Baviskar	21	–	–	21
Timer Driver	Darpan Baviskar	17	–	–	17
MMU Driver	Darpan Baviskar	17	–	–	17
Type Definitions	Harshit	17	–	–	17
Memory Manager	Harshit	45	7	–	52
Status Codes	Piyush Singh Bhati	6	–	–	6
Scheduler	Piyush Singh Bhati	54	14	–	68
Context Switch	Aashma	8	–	–	8
Exception Handling	Aashma	–	4	–	4
Kernel API	Aashma	–	4	–	4
System Boot	Aashma	–	–	6	6
System Init	Aashma	–	–	5	5
System API	Aashma	–	–	5	5
System Benchmark	Aashma	–	–	10	10
System Memory Stress	Aashma	–	–	3	3
System Stability	Aashma	–	–	3	3
Total		185	29	32	246

Requirement Traceability

The table below maps each SRS requirement identifier to the range of test cases that verify it. All requirements are covered by at least one passing test.

Table 8.4: Requirement to Test Case Traceability

Requirement	Test Case IDs (range / list)	Count
FR-001	ST-BOOT-001 – ST-INIT-001 (5 cases)	5
FR-002	UT-MMU-001 – ST-BENCH-007 (18 cases)	18
FR-003	UT-MMU-013, UT-MMU-015, CT-KAPI-004	3
FR-004	CT-KAPI-001, ST-BOOT-006, ST-API-003	3
FR-005	UT-TIMER-001 – ST-BENCH-005 (20 cases)	20
FR-012	UT-TIMER-007 – ST-BENCH-006 (5 cases)	5
FR-016	UT-MEM-001 – ST-BENCH-010 (18 cases)	18
FR-017	UT-MEM-016 – ST-MEM-STRESS-002 (9 cases)	9
FR-018	UT-MEM-008 – CT-MEM-007 (19 cases)	19
FR-019	UT-MEM-024 – ST-MEM-STRESS-003 (6 cases)	6
FR-020	UT-MEM-004 – ST-BENCH-009 (17 cases)	17
FR-021	UT-UART-001 – ST-API-001 (10 cases)	10
FR-022	UT-UART-009 – ST-API-001 (15 cases)	15
NFR-001	UT-TYPES-001 – UT-TYPES-017 (17 cases)	17
NFR-002	UT-STAT-001 – UT-STAT-006 (6 cases)	6
SRS-COOP	UT-SCHED-001 – ST-STAB-003 (85 cases)	85

Appendices

Appendix A: Glossary

- **Computation Graph:** Directed acyclic graph representing ML model operations and tensor dependencies in ONNX format.
- **Predictable Execution:** Execution with bounded timing variation and repeatable behavior under consistent conditions.
- **Static Scheduling:** Execution order determined before runtime based on graph analysis.
- **Whole-Graph Optimization:** Optimization considering entire computation graph for memory and execution planning.
- **Tensor Lifetime:** Period during execution when a tensor is live and occupies memory, analyzed for reuse opportunities.
- **WCET:** Worst-Case Execution Time, maximum time a task can take under defined conditions.
- **Unikernel:** Single-address-space system combining application and kernel into one executable with minimal overhead.
- **ONNX:** Open Neural Network Exchange format for representing machine learning models.
- **Cooperative Execution:** Execution model where tasks run to completion without pre-emption, yielding control at defined points.

Appendix B: Acronyms

- **SRS:** Software Requirements Specification
- **ML:** Machine Learning
- **OS:** Operating System
- **MMU:** Memory Management Unit
- **SIMD:** Single Instruction Multiple Data
- **NEON:** ARM's SIMD instruction set extension
- **WCET:** Worst-Case Execution Time
- **HAL:** Hardware Abstraction Layer
- **ONNX:** Open Neural Network Exchange
- **DAG:** Directed Acyclic Graph
- **QEMU:** Quick Emulator virtual machine
- **DVFS:** Dynamic Voltage and Frequency Scaling
- **FPU:** Floating-Point Unit

Appendix C: Requirement Templates

- **Functional Requirements:** FR-XXX where XXX is sequential number
- **Performance Requirements:** PR-XXX where XXX is sequential number
- **Predictability Requirements:** PDR-XXX where XXX is sequential number
- **Security Requirements:** SR-XXX where XXX is sequential number
- **Reliability Requirements:** RR-XXX where XXX is sequential number
- **Power Requirements:** PWR-XXX where XXX is sequential number
- **Documentation Requirements:** DOC-XXX where XXX is sequential number
- **Development Constraints:** DC-XXX where XXX is sequential number

- **Compliance Requirements:** COMP-XXX where XXX is sequential number
- **Business Rules:** BR-XXX where XXX is sequential number

Appendix D: Performance Analysis Summary

Research on unikernel performance characteristics:

- **Boot Time:** 20-100ms vs Linux 1-5s (10-50x improvement) [11]
- **Memory Footprint:** 4-32MB vs Linux 64MB+ (2-16x improvement) [2]
- **Security Surface:** 50 syscalls vs 300+ (6x reduction) [1]
- **Deployment Density:** 5-8x more instances per hardware in cloud environments [11]
- **Latency Characteristics:** Reduced variance compared to general-purpose OS scheduling [10]
- **Throughput:** Comparable to Linux with minimal overhead for specialized workloads

Appendix E: Research Basis for Architectural Decisions

- **ARM64 Selection:** ARMv8-A provides architectural benefits including larger register file and advanced SIMD capabilities beneficial for tensor operations [4].
- **Unikernel Benefits:** Library OS approach reduces abstraction overhead and attack surface while maintaining performance for specialized applications [1].
- **Predictable Execution:** Static analysis and scheduling can improve timing predictability for embedded inference applications [6].
- **Performance Monitoring:** Essential for optimization and validation in resource-constrained edge deployments [12].

Appendix F: UART Communication Protocol

- **Baud Rate:** 115200 bps, 8 data bits, no parity, 1 stop bit
- **Message Format:** [START_BYTE][LENGTH][COMMAND][PAYLOAD][CRC]
- **Commands:**
 - 0x01: Load Model (payload: ONNX graph)
 - 0x02: Set Input Tensor (payload: tensor data)
 - 0x03: Run Inference
 - 0x04: Get Results
 - 0x05: System Status
 - 0x06: Configuration Update
- **Response Format:** [START_BYTE][LENGTH][STATUS][PAYLOAD][CRC]
- **Error Handling:** Timeout after 5 seconds, retry up to 3 times

Appendix G: ONNX Operator Support Matrix

Table 8.5: Supported ONNX Operators (Initial Implementation)

Operator	Support Level	Notes
Conv	Stub	2D convolution – implementation pending NEON optimisation
MatMul	Full	Matrix multiplication
Add, Sub, Mul	Full	Element-wise operations
Relu, Sigmoid, Tanh	Full	Activation functions
MaxPool, AveragePool	Stub	2D pooling – implementation pending
Concat	Full	Tensor concatenation
Reshape	Limited	Static shapes only
Transpose	Limited	Fixed permutations
BatchNormalization	Future	Post-training quantization assumed
Dynamic Operators	Excluded	Shape, Resize, etc.

Appendix H: Hardware Platform Specifications

- **Primary Development Platform:** QEMU ARM64 virt machine
 - CPU: Cortex-A53 (or specified via -cpu parameter)
 - Memory: 512MB minimum, 1GB recommended
 - Peripherals: PL011 UART, ARM Generic Timer, GICv2
- **Target Physical Platforms:**
 - Raspberry Pi 3/4 (Cortex-A53/A72)
 - NVIDIA Jetson Nano (Cortex-A57)
 - Generic ARM64 development boards
- **Toolchain Requirements:**
 - GCC 10+ or Clang 12+ with aarch64-linux-gnu target
 - QEMU 5.0+ for system emulation
 - Make or CMake for build system

Bibliography

- [1] A. Madhavapeddy, R. Mortier, C. Rotsos, D. Scott, B. Singh, T. Gazagnaire, S. Smith, S. Hand, and J. Crowcroft, “Unikernels: Library operating systems for the cloud,” in *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2013, pp. 461–472.
- [2] D. E. Williams, “Unikernel Monitors: Extending Minimalism Outside of the Box,” in *8th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 16)*, 2016.
- [3] A. Kivity, D. Laor, G. Costa, P. Enberg, N. Har’El, D. Marti, and V. Zolotarov, “OSv—optimizing the operating system for virtual machines,” in *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, 2014, pp. 61–72.
- [4] ARM Limited, *ARM Architecture Reference Manual ARMv8, for ARMv8-A architecture profile*, ARM DDI 0487G.a, 2023.
- [5] ARM Limited, *ARM NEON Programmer’s Guide*, ARM DEN 0018D, 2023.
- [6] R. I. Davis and A. Burns, “A survey of hard real-time scheduling for multiprocessor systems,” *ACM Computing Surveys*, vol. 43, no. 4, pp. 1–44, Oct. 2013.
- [7] R. Wilhelm, J. Engblom, A. Ermedahl, N. Holsti, S. Thesing, D. Whalley, G. Bernat, C. Ferdinand, R. Heckmann, T. Mitra, F. Mueller, I. Puaut, P. Puschner, J. Staschulat, and P. Stenström, “The worst-case execution-time problem—overview of methods and survey of tools,” *ACM Transactions on Embedded Computing Systems*, vol. 7, no. 3, pp. 1–53, May 2008.
- [8] S. Liu, J. Wei, F. Liang, and K. Xu, “A survey on security of machine learning systems: Attacks, defenses and evaluation,” *arXiv preprint arXiv:2103.05345*, 2021.
- [9] L. Merritt and A. Sasan, “Low-power design techniques for edge AI accelerators,” in *2022 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2022, pp. 1–5.
- [10] F. Manco, C. Lupu, F. Schmidt, J. Mendes, S. Kuenzer, S. Sati, K. Yasukata, C. Raiciu, and F. Huici, “My VM is lighter (and safer) than your container,” in *Proceedings of the 26th Symposium on Operating Systems Principles*, 2017, pp. 218–233.
- [11] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, “Firecracker: Lightweight virtualization for serverless applications,” in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, 2020, pp. 419–434.
- [12] A. S. Tanenbaum and A. S. Woodhull, *Operating Systems: Design and Implementation*, 3rd ed. Pearson Education, 2006.